Simularea monitorizării nivelului de poluanți din aer cu avertizare sonoră

INTELIGENȚĂ ARTIFICIALĂ
ȘI SISTEME EXPERT

Student: Boșnegeanu Adrian-Gabriel
Cadru didactic îndrumător: Dr.Ing. Marius DOBREA

Boșnegeanu Adrian-Gabriel
AIA IV , Gr.1 , FH , UTCB

CUPRINS

I. INTRODUCERE_____3
II. TEHNOLOGII FOLOSITE_____3
III. DESCRIEREA MODULUI DE FUNCȚIONARE_____4

Boșnegeanu Adrian-Gabriel
AIA IV , Gr.1 , FH , UTCB

Simularea monitorizării nivelului de poluanți din aer cu avertizare sonoră

I. Introducere

În cadrul proiectului se realizează o aplicație software ce simulează citirea de la senzori a concentrațiilor de poluanți din aer și alertează utilizatorul printr-un semnal sonor atunci când valorile măsurate depășesc limitele de siguranță. Aplicația permite vizualizarea datelor într-o interfață grafică.

II. Tehnologii folosite

Aplicația software este dezvoltată în Python cu ajutorul următoarelor librării:

- Python: Limbajul de programare folosit pentru implementarea logicii aplicației.
- Pandas: Biblioteca pentru manipularea și procesarea datelor (de exemplu, citirea fișierelor CSV și manipularea DataFrame-urilor).
- NumPy: Folosit pentru manipularea eficientă a datelor numerice.
- Tkinter: Biblioteca pentru crearea interfeței grafice a utilizatorului (GUI).
- Matplotlib: Folosit pentru generarea, afișarea și salvarea graficelor în format PDF.
- Winsound: Folosit pentru a emite semnale sonore de avertizare.

```
import pandas as pd
import numpy as np
from tkinter import Tk, Text, Button, Scrollbar
import matplotlib.pyplot as plt
import winsound
from matplotlib.backends.backend_pdf import PdfPages
```

Fig II.1. Antetul codului

III. Descrierea modului de lucru

Se va simula citirea în timp real a valorilor de poluanți PM1, PM10, PM2.5 pe baza unor citiri anterioare stocate în fișiere de tip .csv pe structura reprezentată mai jos.

time,	latitude,	longitude,	altitude,	pm1
1539810018,	47.6833,	22.4667,	0,	7.2
1539810078,	47.6833,	22.4667,	0,	9.6
1539810138,	47.6833,	22.4667,	0,	9.2
1539810198,	47.6833,	22.4667,	0,	8.4
1539810258,	47.6833,	22.4667,	0,	9.2
1539810318,	47.6833,	22.4667,	0,	10.4
1539810378,	47.6833,	22.4667,	0,	10.8
1539810438,	47.6833,	22.4667,	0,	9.6
1539810498,	47.6833,	22.4667,	0,	8

Fig III.2. Structură .CSV

Astfel, se vor datele din fiecare fișier .csv în dataframe-uri și se vor concatena astfel încât să dispunem de toate datele necesare într-un singur dataframe. De asemenea, după cum apare și în (Fig III.1.), asupra coloanei „time” se realizează o conversie din formatul actual (UNIX) în format clasic.

```
def readData():  1 usage
    global df_final  # Indica faptul ca modifici variabila globala
    dfPM1 = pd.read_csv(r"C:\Users\bosne\Desktop\Python AI\Carei Satu Mare\Urad PM1.csv")
    dfPM10 = pd.read_csv(r"C:\Users\bosne\Desktop\Python AI\Carei Satu Mare\Urad PM10.csv")
    dfPM25 = pd.read_csv(r"C:\Users\bosne\Desktop\Python AI\Carei Satu Mare\Urad PM25.csv")
    #concateneaza toate DataFrame-urile
    df_concat = pd.concat( objs: [dfPM1, dfPM10, dfPM25], axis=1)
    #Elimina coloanele duplicate si creeaza o copie completa
    df_final = df_concat.loc[:, ~df_concat.columns.duplicated()].copy()
    #convertire timp UNIX
    df_final['time'] = pd.to_datetime(df_final['time'], unit='s')
```

Fig III.3. Secțiune de cod destinată citirii datelor și conversiei

În cadrul următoarei funcții din codul sursă, se va simula o citire de date în timp real preluând datele din dataframe-ul creat și stocându-le într-un array numpy cu ajutorul căruia vom realiza graficul fluctuației în timp a concentrațiilor de poluanți ulterior.

```
def simRead():  2 usages
    if not pauza:  # Continua doar daca simularea nu este intrerupta
        if indexSim < len(df_final):
            nowTime = df_final["time"].iloc[indexSim]
            nowPM1 = df_final["pm1"].iloc[indexSim]
            nowPM10 = df_final["pm10"].iloc[indexSim]
            nowPM25 = df_final["pm25"].iloc[indexSim]

            # Afisare in text box
            textbox.config(state="normal")
            textnou = f"Time: {nowTime}, PM1: {round(nowPM1, 1)}, PM10: {round(nowPM10, 1)}, PM2.5: {round(nowPM25, 1)}\n"
            start_index = textbox.index("end-1c linestart")
            textbox.insert( index: "end", textnou)
            # tinem textboxul scrollat
            textbox.yview_pickplace("end")
            # highlight si alarma pentru valori prea ridicate
            if nowPM1 > 100:
                textbox.tag_add( tagName: "highlight_red",  index1: f"{start_index}+27c",  *args: f"{start_index}+37c")
                winsound.Beep( frequency: 1000,  duration: 300)
            elif nowPM1>50:
                    textbox.tag_add( tagName: "highlight_yellow",  index1: f"{start_index}+27c",  *args: f"{start_index}+37c")
            if nowPM10 > 120:
                textbox.tag_add( tagName: "highlight_red",  index1: f"{start_index}+38c",  *args: f"{start_index}+49c")
                winsound.Beep( frequency: 1000,  duration: 300)
            elif nowPM10>100:
                textbox.tag_add( tagName: "highlight_yellow",  index1: f"{start_index}+38c",  *args: f"{start_index}+49c")
            if nowPM25 > 100:
                textbox.tag_add( tagName: "highlight_red",  index1: f"{start_index}+51c",  *args: f"{start_index}+60c")
                winsound.Beep( frequency: 1000,  duration: 300)
            elif nowPM25>50:
                textbox.tag_add( tagName: "highlight_yellow",  index1: f"{start_index}+51c",  *args: f"{start_index}+60c")


            textbox.config(state="disabled")

            # Stocare in array NumPy
            new_entry = np.array( object: [[nowTime, nowPM1, nowPM10, nowPM25]], dtype=object)
            dateSim = np.vstack([dateSim, new_entry])
            indexSim += 1
        # viteza simulare
        fereastra.after( ms: 10, simRead)
```

.Fig III.4. Funcția simRead

După memorarea valorilor curente le afișăm în interfața grafică realizată cu ajutorul biblotecii tkinter prin intermediul unui textbox. Verificăm nivelul fiecărui poluant și în cazul în care se reperează valori anormale le vom evidenția printr-un highlight galben. Depistarea unor valori ce prezintă un pericol pentru sănătate, se folosește un highlight roșu pentru evidențierea acestora și prin intermediul biblotecii winsound se emite un sunet de tip „beep” de 0,3 secunde.

Următorul pas în structura codului (Fig. III.5.) reprezintă implementarea funcțiilor de pornire și de punere pe pauză a simulării. În cadrul acestor funcții folosim variabila globală „pauza” pentru a verifica și transmite starea de rulare a simulării.

```python
# incepere simularii
def startSim():  1 usage
    global pauza
    pauza = False  # Reia simularea daca este intrerupta
    button_start.config(state="disabled")  # Dezactiveaza butonul de start
    button_pauza.config(state="normal")  # Activeaza butonul de pauza
    simRead()
    button_plot.config(state="disabled")

# Pauza simulare
def pauseSim():  1 usage
    global pauza
    pauza = True  # intrerupe simularea
    button_start.config(state="normal")  # Activeaza butonul de start
    button_pauza.config(state="disabled")  # Dezactiveaza butonul de pauza
    button_plot.config(state="normal")
```

Fig III.5. Funcțiile de pornire și pauză ale simulării

Pentru trasarea și afișarea graficului variației poluanților, în timp utilizăm metode oferite de biblioteca matplotlib. Pentru timp ne folosim de prima coloană a array-ului dateSim și valorile poluanților le vom extrage din coloanele următoare.

```python
# Reprezentarea grafica a datelor stocate
def plotData():  1 usage
    if dateSim.size > 0:
        times = dateSim[:, 0]
        pm1_values = dateSim[:, 1].astype(float)
        pm10_values = dateSim[:, 2].astype(float)
        pm25_values = dateSim[:, 3].astype(float)

        plt.figure(figsize=(10, 6))
        plt.plot( *args: times, pm1_values, label="PM1")
        plt.plot( *args: times, pm10_values, label="PM10")
        plt.plot( *args: times, pm25_values, label="PM2.5")

        plt.xlabel("Timp")
        plt.ylabel("Concentratie (µg/m³)")
        plt.title("Evolutia concentratiilor de PM1, PM10 si PM2.5")
        plt.legend()
        plt.grid()
        plt.tight_layout()

        plt.show()
        button_saveplot.config(state="normal")
    else:
        print("Nu exista date de afisat in acest moment.")
```

Fig. III.6. Funția de afișare a graficului

Similar, pentru salvarea în format PDF a graficului, îl creem prin biblioteca matplotlib și îl salvăm cu ajutorul PDFPages în funcția savePlot.

```
def savePlot():  1 usage
    times = dateSim[:, 0]
    pm1_values = dateSim[:, 1].astype(float)
    pm10_values = dateSim[:, 2].astype(float)
    pm25_values = dateSim[:, 3].astype(float)

    plt.figure(figsize=(10, 6))
    plt.plot( *args: times, pm1_values, label="PM1")
    plt.plot( *args: times, pm10_values, label="PM10")
    plt.plot( *args: times, pm25_values, label="PM2.5")

    plt.xlabel("Timp")
    plt.ylabel("Concentratie (µg/m³)")
    plt.title("Evolutia concentratiilor de PM1, PM10 si PM2.5")
    plt.legend()
    plt.grid()
    plt.tight_layout()
    locatiePDF = r"C:\Users\bosne\Desktop\grafic_poluanti.pdf"
    with PdfPages(locatiePDF) as pdf:
        pdf.savefig()
```

Fig. III.7. Funcția de salvare PDF a graficului

În ultima parte a codului sursă apelăm funcția de citire a datelor din fișierele .csv, definim o fereastră pentru a ne configura interfața grafică și stabilim proprietățile controalelor de care avem nevoie:

- Textbox-ul (pentru afișare)
- Butonul de start al simulării
- Butonul de întrerupere a simulării
- Butonul de afișare a graficului
- Butonul de salvare a graficului

```
# Citirea datelor
readData()

# crearea ferestrei Tkinter
fereastra = Tk()
fereastra.title("Analiza poluanti")
fereastra.geometry("800x600")

# crearea unui text box pentru afisare
textbox = Text(fereastra, wrap="word", width=90, height=25)
textbox.pack(pady=10)

#configuram taguri pentru evidentiere in textbox
textbox.tag_configure("highlight_yellow", background="yellow")
textbox.tag_configure("highlight_red", background="red")
# Crearea butoanelor
button_start = Button(fereastra, text="Porneste simularea", command=startSim)
button_start.pack(pady=10)

button_pauza = Button(fereastra, text="Pune pe pauza", command=pauseSim, state="disabled")
button_pauza.pack(pady=10)

button_plot = Button(fereastra, text="Reprezinta grafic", command=plotData, state="disabled")
button_plot.pack(pady=10)

button_saveplot = Button(fereastra, text="Salveaza grafic", command=savePlot, state="disabled")
button_saveplot.pack(pady=10)

# Afisare fereastra
fereastra.mainloop()
```

Fig. III.8. Instrucțiuni efective din codul sursă

BIBLIOGRAFIE

- Cursuri și lucrări de laborator
- Python Docs (https://docs.python.org/)
- StackOverflow (https://stackoverflow.com)